Grounded SAM 2 — local evaluation on NVIDIA GeForce RTX 3080

Pipeline: **text prompt -> Grounding DINO 1.0 (local) -> box -> SAM 2.1 (local) -> mask / video track**. Fully local. No cloud API, no API key, no image upload, no network at inference.

> **Bottom line — classification: PROMISING, but only as a category-prompted detect-and-track pipeline. As a referring-expression system on aerial imagery it does not work: top-1 localization is 22% at IoU>=0.5.** Full reasoning in section 15.

1. What was actually run

Phase	What	Status
1	Smoke test, single aerial image, prompts `car` / `person`	PASS
2	RefDrone sample, 50 (image, expression) pairs, seed 1234	PASS (ran; accuracy is poor — see 5)
3	Video tracking, 2 clips, one-time acquisition + SAM 2 propagation	PASS (found a real failure mode — see 6)
4	Benchmark, config A (SAM2.1 Tiny) vs B (SAM2.1 Small)	PASS

Nothing here is claimed on the basis of a script exiting 0. Every result below was checked against the rendered masks or the per-frame mask geometry.

2. Environment

GPU	NVIDIA GeForce RTX 3080 (sm_86, 11911 MB), driver 580.95.05
CPU / RAM	12th Gen Intel(R) Core(TM) i9-12900K / 62.5 GB
Python / torch	3.12.3 / 2.5.1+cu124 (CUDA 12.4)
transformers	4.44.2
GD CUDA extension	**COMPILED**
Cloud SDK installed	no

`TORCH_CUDA_ARCH_LIST=8.6` was **detected from the GPU**, not guessed.

The custom `MultiScaleDeformableAttention` CUDA op had to be compiled from source. This matters more than it looks: upstream's fallback is gated on `torch.cuda.is_available()`, **not** on whether the

extension loaded. So without the extension Grounding DINO does not quietly degrade to a slower CPU path on a GPU box — it crashes. There is no silent-CPU-fallback risk here, and no CPU timing is reported as if it were GPU. `gsam2_local.device()` raises rather than run on CPU.

Getting `nvcc` without sudo and without touching system CUDA required assembling an isolated toolchain (micromamba prefix + pinned CUDA 12.4 math headers). System CUDA and global Python were not modified.

---

3. Local-only verification

- Cloud SDK (`dds_cloudapi_sdk`): **not installed**.
- `assert_no_cloud_imports()` runs after every inference call and raises if any `dds_cloudapi` / `dinox` module is ever present in `sys.modules`. It never fired.
- The 6 upstream demos that require the DDS cloud API were enumerated and avoided (listed in `configs/local_eval.yaml`).
- **No upstream submodule file was modified.** All work lives in a wrapper, `scripts/gsam2_local.py` .

---

4. Network dependency — the pipeline was NOT air-gap clean by default

This was measured with `strace -f -e trace=connect` , not assumed.

An **unguarded** run opens real outbound connections: DNS, then `:443` to `huggingface.co` (CloudFront). Cause: Grounding DINO's text encoder calls `from_pretrained("bert-base-uncased")` , and transformers **revalidates the cached files against the hub on every model load** — even though the weights are already on disk. No imagery is uploaded (it is a metadata fetch), but the pipeline is not network-free out of the box, which is a blocker for an air-gapped target.

Fixed in the adapter: `gsam2_local` sets `HF_HUB_OFFLINE` / `TRANSFORMERS_OFFLINE=1` at import, before transformers loads. Re-verified with strace: **zero `AF_INET` `connect()` syscalls**, identical detections. Offline enforcement is now the default.

One consequence worth carrying to Jetson: **the BERT text encoder is not in `checkpoints/` .** It lives in `~/.cache/huggingface` (~421 MB) and must be staged separately on any air-gapped device, or Grounding DINO will not load at all.

---

5. Phase 2 — RefDrone referring expressions (the headline negative result)

50 pairs, seed 1234, split 30 train / 10 val / 10 test. Config: GD Swin-T + SAM2.1 tiny, box_thr 0.25, text_thr 0.2.

Metric	Value
Top-1 localization, IoU>=0.5	**22.0%** (11/50)
Mean best IoU	0.212
Median best IoU	0.004
No-detection rate	4.0%
Multiple-detection rate	84.0% (median 4 boxes, max 25)
Mean GD latency	104 ms
Mean SAM 2 latency	26 ms

What is actually failing. Grounding DINO resolves the category in the expression and largely **ignores the referring qualifiers**. `"The black cars near the intersection."` returns 10 boxes and the top-1 lands on a dark car that is not the referent (IoU 0.00) — while SAM 2 segments that wrong box perfectly. The failure is in language grounding, not segmentation. A clean, confident mask on the wrong object is the characteristic output. (figures 02, 05)

The same expression, `"The blue vehicles in the image."`, scores **IoU 0.84 on one frame and 0.00 on another** (figures 01, 03). The prompt is not the controlling variable; the scene is.

84% of expressions return more than one box. For a pipeline that must hand one box to SAM 2, picking the highest-confidence box is close to arbitrary when the detector has found 8 instances of the right class and cannot tell which one the sentence means.

Metric caveats — read these before quoting 22%. - This is my protocol, not RefDrone's official metric: highest-confidence box, scored as max IoU over the GT boxes for that expression. **Do not compare it to published RefDrone numbers.** I did not run the official evaluator. - It is the protocol that matters for this pipeline, though, because SAM 2 must be seeded with exactly one box. - RefDrone contains multi-target expressions. Top-1 gives no credit for finding 24 of 25 referenced objects and ranking the wrong one first, so it is strict on those. - **RefDrone provides bounding boxes only — no mask ground truth.** SAM 2's mask quality is therefore assessed visually and is deliberately **not scored**. No segmentation ground truth was invented.

---

6. Phase 3 — video tracking, and the failure mode that matters

Protocol: Grounding DINO runs **once** on the first frame; SAM 2 propagates with **no re-detection**. This isolates acquisition cost from steady-state tracking cost, and it is what makes the architecture cheap.

6a. `zebra.mp4` (1280x720, 200 frames, prompt `zebra`) — target-present control

- **200/200 frames masked, 0 empty, 0 dropouts, 0 identity switches.**
- Survived a mutual-occlusion event at f28-30, where another zebra walked in front of the target and mask area fell to 40%; SAM 2 wrapped the mask around the occluder and recovered the full silhouette.
- Two transient mask-fragmentation artefacts (a sliver bleeding onto an adjacent animal), self-corrected within 1-2 frames.

- **Not claimed:** that identity was preserved through the f29 occlusion. Five near-identical, mutually-occluding animals and no instance ground truth in the clip — it is not certifiable from this data, so I do not certify it.

6b. `tracking_car.mp4` (1920x1080, 200 frames, prompt `car`) — the silent identity switch

Visible runs: `[[0, 68], [176, 199]]`.

- **Frames 0-68: a clean track.** Mask grew smoothly 1,592 -> 70,482 px as the car approached the overpass camera. Zero same-target dropouts, no drift.
- **Frame 69: the target left the frame.** The mask was shrinking and clipped by the `bottom` border — the car drove out underneath the camera. The 107 empty frames that follow are **correct behaviour**, not a tracking failure.
- **Frame 176: SAM 2 silently re-bound object id 1 to a DIFFERENT car**, 953 px away, near the horizon — and reported it with full confidence (figure 07). The original car is gone for good and cannot return.

Why this is the important result. SAM 2 has **no terminal "object is gone" state and no identity check**, and in this architecture the detector never re-runs, so nothing ever corrects the switch. A tracker that quietly swaps targets and keeps reporting is worse than one that reports loss.

It is also a trap for the evaluator. I got this wrong twice before getting it right: counting empty masks called those 107 frames a 53.5% tracking failure; then splitting on last-visible-frame called them dropouts that "recovered". Both are artefacts of scoring mask **area** alone. Only mask **geometry** — was it shrinking, was it clipped by a border, how far did the centroid jump — separates egress from drift from an identity switch. The metric now records per-frame area, bbox and centroid so this is auditable.

A naive empty-mask metric scores this false positive as a successful recovery.

---

7. Phase 4 — performance (NVIDIA GeForce RTX 3080)

3 warm-ups discarded, 10 measured image iterations, 120 propagated frames. Image input 1920x1080 (prompt `car`, 7 boxes); tracking input 1280x720.

	A: GD Swin-T + SAM2.1 Tiny	B: GD Swin-T + SAM2.1 Small
Model load (cold)	2253 ms	2246 ms
First inference (cold)	520 ms	524 ms
Warm image latency	**143 ms (7.0 FPS)**	**146 ms (6.8 FPS)**
of which Grounding DINO	101 ms	101 ms
of which SAM 2	42 ms (enc 21 + dec 22)	45 ms (enc 24 + dec 22)
p95 image latency	145 ms	148 ms
Tracking throughput	**32.5 FPS** (30.8 ms/frame)	**30.3 FPS** (33.0 ms/frame)
SAM 2 video init	2072 ms	2179 ms
Peak VRAM (tracking)	2138 MB	2180 MB
Peak CPU RSS	2786 MB	2785 MB

All GPU regions are timed with `torch.cuda.synchronize()` on **both** sides. No asynchronous kernel-launch time is reported as latency.

Grounding DINO is ~70% of per-image latency and is identical in both configs. Tiny -> Small costs 3 ms/image, 2.1 FPS of tracking and 43 MB of VRAM. **The SAM 2 backbone is not the lever on this pipeline's cost — the detector is.**

---

8. Two measurement bugs I found in my own harness

Recording these because both would have silently flattered the results.

1. **SAM 2 latency was decoder-only.** `sam2_masks()` called `predictor.set_image()` outside the timer. `set_image()` runs the Hiera image encoder — the dominant, backbone-dependent stage; `predict()` is just the mask decoder, which is nearly identical across backbones. The tell was that **SAM2.1 Small benchmarked faster than Tiny** (12.0 vs 21.9 ms), which is impossible. Now timed end-to-end, and Small is correctly slower (45 vs 42 ms). Every SAM 2 latency reported before that fix was too low.
2. **Config B was free-riding on config A's warm process.** Run back-to-back, B appeared to load in 630 ms against A's 1963 ms. Isolated into its own process: 2253 vs 2246 ms. The gap was entirely CUDA-context and page-cache carry-over.

---

9. What works

- Fully local, GPU, no network, no cloud API. Reproducible from `configs/local_eval.yaml`.
- **SAM 2 segmentation quality is excellent** — tight, clean masks, on aerial imagery and video alike. When given a correct box it is not the weak link, ever.
- **Tracking is genuinely good while the target is present**, including through occlusion.
- Real-time-ish tracking (32 FPS) at modest VRAM (2138 MB peak).
- The detect-once-then-track architecture is sound and cheap: acquisition is paid once (358 ms), then every subsequent frame costs only SAM 2.

10. What does not work

- **Referring-expression grounding.** 22% top-1 at IoU>=0.5, mean IoU 0.21. Grounding DINO 1.0 is an open-vocabulary **category** detector; it is not a referring-expression resolver, and it discards the qualifiers that carry the referent. This is a **capability gap, not a tuning problem** — no threshold change fixes it.
- **Disambiguation.** 84% of expressions return multiple boxes; top-1 selection is near-arbitrary among same-class instances.
- **Small aerial targets** erode confidence sharply (`person` tops out at 0.48 vs 0.67 for `car` on the same frame).
- **Silent identity switching** after target egress, with no mechanism to detect or correct it.

11. Not measured / not claimed

- SAM 2 mask **accuracy** is unscored: RefDrone has no mask ground truth and none was invented.
- Identity preservation through the zebra occlusion: not certifiable, not claimed.
- No comparison against published RefDrone baselines (different metric — see section 5).
- Jetson performance: **not measured.** Nothing here was run on Jetson or a work laptop.
- Multi-object tracking: only single-object (`obj_id=1`) was exercised.

12. Reproduce

```
source .venv-grounded-sam2/bin/activate
python experiments/grounded_sam2_local/scripts/check_environment.py
python experiments/grounded_sam2_local/scripts/run_image_demo.py \
    --image datasets/refdrone/images/all_image/0000001_02999_d_0000005.jpg \
    --prompts "car" "person"
python experiments/grounded_sam2_local/scripts/run_refdrone_sample.py --seed 1234
python experiments/grounded_sam2_local/scripts/run_video_demo.py \
    --video third_party/grounded_sam_2/assets/tracking_car.mp4 --prompt "car"
python experiments/grounded_sam2_local/scripts/run_video_demo.py \
    --video third_party/grounded_sam_2/assets/zebra.mp4 --prompt "zebra" --start-frame 20
python experiments/grounded_sam2_local/scripts/benchmark_pipeline.py
python experiments/grounded_sam2_local/scripts/summarize_results.py
```

13. Figures

See `figures/`. Detection success (01), the characteristic detection failure (02), prompt ambiguity (03), small aerial targets (04), multiple similar targets (05), tracking success (06), and the identity switch (07).

14. If this continues — what to fix, in order

1. **Replace the language-grounding stage.** It is the bottleneck for both accuracy (22%) and latency (70% of per-image cost). SAM 2 is already good enough.

2. **Add a re-detection / identity-verification trigger.** Any of: mask area collapse, mask clipped by a border, or an N-frame gap should force re-acquisition instead of letting SAM 2 re-bind to whatever appears. This is cheap and directly kills the section-6b failure.
3. Only then worry about the SAM 2 backbone.

---

15. Classification: PROMISING — scoped

Not softened, and not a blanket endorsement:

PROMISING as a category-prompted detect-and-track pipeline. The engineering holds up: fully local, 32 FPS tracking, 2138 MB peak VRAM, excellent segmentation, sound architecture, and it degrades honestly rather than silently falling back to CPU. Prompted with `car` or `person` it works, and the VRAM/throughput envelope fits a Jetson-class target.

NOT USABLE as a referring-expression system. 22% top-1 on RefDrone is not a tuning failure that a threshold sweep will recover — Grounding DINO 1.0 does not resolve referring qualifiers, and the pipeline confidently, cleanly segments the wrong object. If the intended capability is "describe one specific object in natural language and track it," **this pipeline does not deliver that today**, and no amount of SAM 2 tuning will change it, because SAM 2 is not what is failing.

The reason this is PROMISING rather than REFERENCE ONLY: the weak component is the one that is replaceable. The expensive, hard part — real-time memory-based segmentation and tracking that survives occlusion — is the part that works.

---

16. Recommendation for Jetson

Try SAM 2.1 Tiny first. Small buys nothing measurable here: +3 ms per image, **-2.1 FPS** tracking, +43 MB VRAM, and no quality gain observed in any run. Since Grounding DINO dominates latency, spending the Jetson's headroom on the SAM 2 backbone is the wrong trade.

Before any Jetson attempt, note three things this evaluation surfaced: - the **BERT text encoder is not in `checkpoints/`** and must be staged into the HF cache, - the **`MultiScaleDeformableAttention` CUDA extension must be compiled for the Jetson's compute capability**, and it **crashes rather than degrades** if absent, - offline enforcement must be kept on, or the first model load will try to reach `huggingface.co`.

Figure gallery

Figure 01. SUCCESS. 'The blue vehicles in the image.' -> 1 box, IoU 0.84 vs GT. SAM 2 mask is tight on the vehicle.

Figure 02. FAILURE, and the characteristic one. 'The black cars near the intersection.' -> 10 boxes, top-1 IoU 0.00. GD resolved the CATEGORY (cars) and ignored the referring qualifiers ('black', 'near the intersection'). SAM 2 then segmented the wrong box perfectly -- a confident, clean mask on the wrong object.

Figure 03. AMBIGUOUS PROMPT. The SAME expression as figure 01 -- 'The blue vehicles in the image.' -- on a different frame scores IoU 0.00. Identical prompt, 0.84 vs 0.00. The expression is not the controlling variable; the scene is.

Figure 04. SMALL AERIAL TARGETS. 'person' on a 1920x1080 VisDrone aerial frame -> 13 detections, top score 0.48 (vs 0.67 for 'car' on the same frame). Confidence degrades sharply as target size shrinks.

Figure 05. MULTIPLE SIMILAR TARGETS. 'The blue cars park along the street and curbside.' -> 8 boxes for 8 GT objects, yet top-1 IoU 0.01. The detector finds the right NUMBER of the right class and still cannot pick the referent. Taking the highest-confidence box is close to arbitrary here.

Figure 06. TRACKING SUCCESS. zebra.mp4, frame 199 of 200. One-time GD acquisition on frame 0, then SAM 2.1 Tiny propagation with NO re-detection. 200/200 frames masked, zero empty, coherent single-animal mask, survived a mutual-occlusion event at f28-30.

Figure 07. TRACKING FAILURE -- the important one. tracking_car.mp4 frame 176. The car GD acquired on frame 0 drove out of the BOTTOM of the frame at f69 and is gone for good. At f176 SAM 2 silently re-bound object id 1 to a DIFFERENT car 953 px away, near the horizon, and reports it with full confidence. SAM 2 has no terminal 'object is gone' state and no identity check, and the detector never re-runs, so nothing corrects it. A naive empty-mask metric scores this false positive as a successful RECOVERY.